

1 Импорт библиотек

```
# Импорт библиотек
import pandas as pd
import numpy as np

# Визуализация
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    confusion_matrix
)
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from catboost import CatBoostClassifier
from sklearn.model_selection import GridSearchCV
```

2 Первичный анализ данных

```
df = pd.read_excel("для мл.xlsx")
```

```
df.head()
```

	Дата смены	Номер смены	Буровой станок	Парк Б.С.
КИО \				
0	2025-05-08	1	ATLAS COPCO PV-351 №53	ATLAS COPCO PV-351
0.704				
1	2025-05-08	2	ATLAS COPCO PV-351 №53	ATLAS COPCO PV-351
0.704				
2	2025-05-09	2	ATLAS COPCO PV-351 №53	ATLAS COPCO PV-351
0.704				
3	2025-05-09	1	ATLAS COPCO PV-351 №53	ATLAS COPCO PV-351
0.704				
4	2025-05-10	1	ATLAS COPCO PV-351 №53	ATLAS COPCO PV-351

0.704

	КНП	КТГ	Пробурено факт, м	Длительность бурения, ч	Простой, ч \
0	0.824	0.862	188.00	6.75	4.243
1	0.824	0.862	353.69	9.39	0.446
2	0.824	0.862	318.00	8.66	1.640
3	0.824	0.862	415.64	9.48	1.207
4	0.824	0.862	332.00	9.86	0.574

	Очищенная производительность м/ч
0	27.851852
1	37.666667
2	36.720554
3	43.843882
4	33.671400

df.info()

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 17215 entries, 0 to 17214  
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	Дата смены	17215 non-null	datetime64[ns]
1	Номер смены	17215 non-null	int64
2	Буровой станок	17215 non-null	object
3	Парк Б.С.	17215 non-null	object
4	КИО	17215 non-null	float64
5	КНП	17215 non-null	float64
6	КТГ	17215 non-null	float64
7	Пробурено факт, м	17215 non-null	float64
8	Длительность бурения, ч	17215 non-null	float64
9	Простой, ч	17215 non-null	float64
10	Очищенная производительность м/ч	17215 non-null	float64

dtypes: datetime64[ns](1), float64(7), int64(1), object(2)

memory usage: 1.4+ MB

```
print(f"Размер датасета: {df.shape}")
```

Размер датасета: (17215, 11)

df.describe()

	Дата смены	Номер смены	КИО
КНП \			

count	17215	17215.000000	17215.000000
17215.000000			
mean	2025-07-07 03:25:41.423177472	1.49788	0.704885
0.838753			
min	2025-01-01 00:00:00	1.000000	0.471000
0.824000			
25%	2025-04-12 00:00:00	1.000000	0.706000
0.826000			
50%	2025-07-10 00:00:00	1.000000	0.713000
0.828000			
75%	2025-10-02 00:00:00	2.000000	0.713000
0.828000			
max	2025-12-31 00:00:00	2.000000	0.763000
0.879000			
std	NaN	0.50001	0.062668
0.021466			

	КТГ	Пробурено факт, м	Длительность бурения, ч
Простой, ч \			
count	17215.000000	17215.000000	17215.000000
17215.000000			
mean	0.900352	191.560176	7.959025
2.046652			
min	0.862000	24.000000	0.090000
0.000000			
25%	0.905000	126.500000	7.110000
0.500000			
50%	0.905000	182.000000	8.460000
1.286000			
75%	0.908000	242.000000	9.300000
2.872000			
max	0.908000	1072.540000	11.420000
11.000000			
std	0.014359	89.884122	1.910189
2.155314			

	Очищенная производительность м/ч
count	17215.000000
mean	23.953152
min	3.172589
25%	17.446783
50%	23.152966
75%	28.882944
max	88.636170
std	9.065047

df.isnull().sum()

Дата смены	0
Номер смены	0

```

Буровой станок      0
Парк Б.С.           0
КИО                  0
КНП                  0
КТГ                  0
Пробурено факт, м    0
Длительность бурения, ч  0
Простой, ч           0
Очищенная производительность м/ч  0
dtype: int64

```

```
print("Количество дубликатов:", df.duplicated().sum())
```

Количество дубликатов: 0

```

print("Количество буровых станков:", df['Буровой станок'].nunique())
print("Количество парков:", df['Парк Б.С.'].nunique())

```

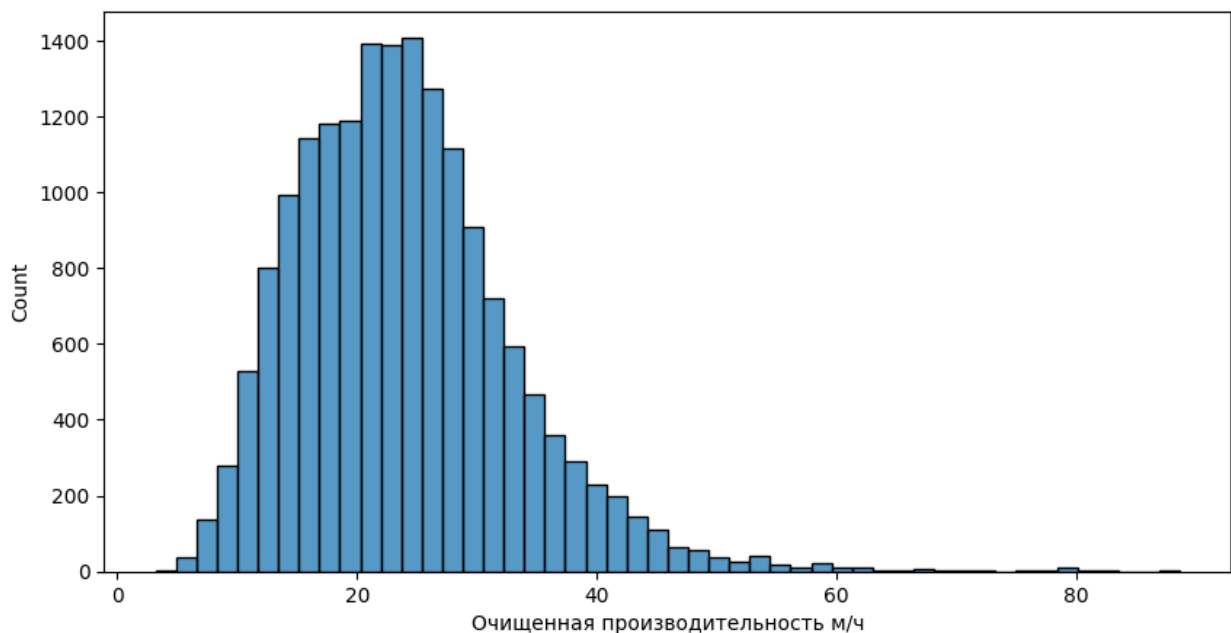
Количество буровых станков: 32

Количество парков: 6

```

plt.figure(figsize=(10,5))
sns.histplot(df['Очищенная производительность м/ч'], bins=50)
plt.show()

```



```

df.groupby('Буровой станок')['Очищенная производительность м/ч'].agg([
    'count',
    'mean',
    'median',
    'min',

```

```
'max'  
]).sort_values('mean')
```

	count	mean	median	min
max				
Буровой станок				
СБШ-250 №53 30.269710	119	17.914448	17.997978	6.983240
СБШ-250 №54 35.099342	647	19.599617	19.487751	5.263158
СБШ-215 №34 35.590319	111	19.656840	19.405815	8.353511
СБШ-250 №30 32.535028	635	19.909273	20.077973	6.112054
СБШ-250 №31 34.704257	358	20.270391	20.867903	6.684492
СБШ-250 №42 36.905842	343	20.290220	20.207254	5.881567
СБШ-215 №40 37.663666	699	21.051893	21.149897	4.675573
СБШ-250 №47 40.427413	583	21.295748	21.112158	5.261910
СБШ-250 №50 34.795062	528	21.340821	21.739925	4.802260
СБШ-250 №46 37.838174	705	21.470206	21.468144	3.172589
СБШ-250 №43 44.556468	565	21.584354	21.297297	5.036726
ATLAS COPCO PV-235 №57 37.446176	680	21.764163	21.752778	4.876463
ATLAS COPCO PV-235 №55 40.114943	351	21.879605	21.808511	6.966774
СБШ-215 №55 41.285907	673	22.372491	22.195122	5.400000
СБШ-215 №35 37.575000	581	22.775274	22.871795	6.974638
СБШ-215 №36 36.494640	714	22.837339	23.404324	6.770833
ATLAS COPCO DML №75 41.274183	658	22.935430	22.505796	6.383305
СБШ-250 №37 42.973990	376	23.267009	23.284365	6.578947
СБШ-215 №49 55.017973	214	23.322801	21.831834	7.649686
ATLAS COPCO DML №77 46.251767	651	23.371603	22.833563	5.377095
СБШ-250 №45 40.389644	697	23.402715	22.945205	5.701754
СБШ-215 №38	552	23.435248	23.026671	8.556548

43.524929					
СБШ-215 №51	416	23.493576	23.437464	5.852417	
44.566661					
СБШ-250 №44	697	24.765021	24.137931	5.902384	
53.523203					
ATLAS COPCO DML №73	655	25.148913	23.783784	5.768444	
61.186074					
СБШ-215 №48	686	25.530781	25.132660	6.436782	
53.045638					
СБШ-215 №52	719	25.919926	25.358540	7.360673	
50.229440					
ATLAS COPCO DML №74	616	26.016682	25.111000	5.401235	
62.436973					
ATLAS COPCO DML №76	614	26.807058	26.554433	6.029412	
54.252170					
ATLAS COPCO PV-351 №52	677	35.360766	36.143040	11.279826	
59.025715					
ATLAS COPCO PV-351 №53	446	35.787073	36.543427	13.627354	
79.111989					
ATLAS COPCO PV-351 №51	249	41.597903	40.632054	15.193370	
88.636170					

Проведён анализ распределения производительности по каждому буровому станку.

Полученные значения используются для понимания особенностей работы оборудования и последующего формирования индивидуальных порогов риска.

3 Формирование целевой переменной

```
station_p25 = df.groupby('Буровой станок')['Очищенная
производительность м/ч'].quantile(0.25)
```

```
station_p25.head()
```

Буровой станок

ATLAS COPCO DML №73 17.493972

ATLAS COPCO DML №74 17.803646

ATLAS COPCO DML №75 16.813230

ATLAS COPCO DML №76 20.280058

ATLAS COPCO DML №77 16.457624

Name: Очищенная производительность м/ч, dtype: float64

В качестве порогового значения риска используется 25-й перцентиль производительности каждого бурового станка.

Такой подход позволяет оценивать снижение производительности относительно собственной исторической эффективности станка, а не сравнивать различные модели оборудования между собой.

```
df['p25'] = df['Буровой станок'].map(station_p25)
df[['Буровой станок', 'Очищенная производительность м/ч',
'p25']].head()
```

	Буровой станок	Очищенная производительность м/ч	p25
0	ATLAS COPCO PV-351 №53	27.851852	27.958771
1	ATLAS COPCO PV-351 №53	37.666667	27.958771
2	ATLAS COPCO PV-351 №53	36.720554	27.958771
3	ATLAS COPCO PV-351 №53	43.843882	27.958771
4	ATLAS COPCO PV-351 №53	33.671400	27.958771

```
df['risk'] = (df['Очищенная производительность м/ч'] <
df['p25']).astype(int)

df['risk'].value_counts(normalize=True) * 100
```

```
risk
0    74.975312
1    25.024688
Name: proportion, dtype: float64
```

Доля наблюдений с риском составила около 25%.

Полученное распределение является ожидаемым, поскольку риск определяется через 25-й перцентиль производительности каждого станка.

```
df[['Буровой станок', 'Очищенная производительность м/ч', 'p25',
'risk']].sample(20)
```

	Буровой станок	Очищенная производительность м/ч
p25 \		
12796	СБШ-215 №52	24.119449
19.358681		
131	ATLAS COPCO PV-351 №53	38.763298
27.958771		
15761	СБШ-215 №38	26.572668
18.306972		
15212	СБШ-215 №40	33.240535
15.462084		
15745	СБШ-215 №38	25.583333
18.306972		
5917	СБШ-250 №54	18.141593
15.140236		
16263	СБШ-215 №36	23.582766
18.128426		
16967	СБШ-215 №35	21.554404
18.061674		
16313	СБШ-215 №36	23.769575
18.128426		
9539	СБШ-250 №44	12.133183

19.326065		
15842	СБШ-215 №36	35.032895
18.128426		
2011	ATLAS COPCO PV-235 №57	16.318538
16.745583		
15040	СБШ-215 №40	31.588448
15.462084		
14178	СБШ-215 №48	34.022989
18.578138		
7156	СБШ-250 №47	10.898204
15.547499		
7642	СБШ-250 №46	34.848485
16.315422		
12398	СБШ-215 №55	29.230769
16.510319		
16357	СБШ-215 №36	14.665653
18.128426		
5617	СБШ-250 №54	24.709677
15.140236		
16997	СБШ-215 №35	25.724638
18.061674		

	risk
12796	0
131	0
15761	0
15212	0
15745	0
5917	0
16263	0
16967	0
16313	0
9539	1
15842	0
2011	1
15040	0
14178	0
7156	1
7642	0
12398	0
16357	1
5617	0
16997	0

```
df.groupby('risk')[['КИО', 'КНП', 'КТГ', 'Простой, ч']].mean()
```

	КИО	КНП	КТГ	Простой, ч
risk				
0	0.704882	0.838750	0.900351	1.949045
1	0.704893	0.838762	0.900356	2.339089

Наиболее заметное различие между классами наблюдается по показателю простоя.

Для наблюдений с риском средний простой выше.

```
df.groupby('Номер смены')['risk'].mean() * 100
```

Номер смены	
1	23.403517
2	26.659666

Name: risk, dtype: float64

Во вторую смену доля наблюдений с риском несколько выше, чем в первую смену.

```
df.groupby('risk')['Простой, ч'].mean()
```

risk	
0	1.949045
1	2.339089

Name: Простой, ч, dtype: float64

Средний простой при наличии риска составляет 2,34 часа против 1,95 часа при отсутствии риска.

Это подтверждает влияние простоев на вероятность снижения производительности.

4 Подготовка данных для машинного обучения

```
features = [  
    'Буровой станок',  
    'Номер смены',  
    'КИО',  
    'КНП',  
    'КТГ',  
    'Простой, ч'  
]  
  
target = 'risk'  
  
X = df[features]  
y = df[target]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42,
```

```
) stratify=y
```

5 Обучение и сравнение моделей

Для решения задачи классификации были выбраны три модели машинного обучения.

Логистическая регрессия используется в качестве базовой интерпретируемой модели и позволяет оценить возможность линейного разделения классов.

Random Forest представляет собой ансамблевый алгоритм на основе деревьев решений и позволяет выявлять более сложные нелинейные зависимости между признаками.

CatBoost выбран как современный алгоритм градиентного бустинга, эффективно работающий с категориальными признаками и показывающий высокое качество на табличных данных.

Сравнение моделей будет выполнено по метрикам Accuracy, Precision, Recall, F1-score и ROC-AUC.

5.1 Логистическая регрессия

```
categorical_features = ['Буровой станок', 'Номер смены']  
numeric_features = ['КИО', 'КНП', 'КТГ', 'Простой, ч']
```

Для обучения логистической регрессии требуется предварительная подготовка данных.

Категориальные признаки («Буровой станок», «Номер смены») преобразуются с помощью One-Hot Encoding в набор бинарных признаков.

Числовые признаки («КИО», «КНП», «КТГ», «Простой, ч») масштабируются с использованием StandardScaler, что позволяет привести их к единому масштабу и повысить устойчивость линейной модели.

```
preprocessor_lr = ColumnTransformer(  
    transformers=[  
        ('cat', OneHotEncoder(handle_unknown='ignore'),  
         categorical_features),  
        ('num', StandardScaler(), numeric_features)  
    ]  
)
```

Для объединения этапов предобработки и обучения используется Pipeline.

Такой подход позволяет автоматически выполнять одинаковые преобразования как для обучающей, так и для тестовой выборки, а также исключает риск ошибок при подготовке данных.

```

logreg_model = Pipeline(
    steps=[
        ('preprocessor', preprocessor_lr),
        ('model', LogisticRegression(
            max_iter=1000,
            class_weight='balanced',
            random_state=42
        ))
    ]
)

logreg_model.fit(X_train, y_train)

Pipeline(steps=[('preprocessor',
                  ColumnTransformer(transformers=[('cat',
                                                  OneHotEncoder(handle_unknown='ignore'),
                                                  ['Буровой станок',
                                                  'Номер смены']),
                                                  ('num',
                                                  StandardScaler(),
                                                  ['КИО', 'КНП',
                                                  'КТГ',
                                                  'Простой, ч'])])),
                ('model',
                 LogisticRegression(class_weight='balanced',
                                    max_iter=1000,
                                    random_state=42))]))

y_pred_lr = logreg_model.predict(X_test)
y_proba_lr = logreg_model.predict_proba(X_test)[: , 1]

logreg_metrics = {
    'Модель': 'Logistic Regression',
    'Accuracy': round(accuracy_score(y_test, y_pred_lr), 3),
    'Precision': round(precision_score(y_test, y_pred_lr), 3),
    'Recall': round(recall_score(y_test, y_pred_lr), 3),
    'F1-score': round(f1_score(y_test, y_pred_lr), 3),
    'ROC-AUC': round(roc_auc_score(y_test, y_proba_lr), 3)
}

print(classification_report(y_test, y_pred_lr))

```

	precision	recall	f1-score	support
0	0.77	0.69	0.73	2581
1	0.30	0.39	0.34	862
accuracy			0.61	3443
macro avg	0.53	0.54	0.53	3443

weighted avg	0.65	0.61	0.63	3443
--------------	------	------	------	------

Логистическая регрессия показала ограниченную способность выявлять наблюдения с риском снижения производительности.

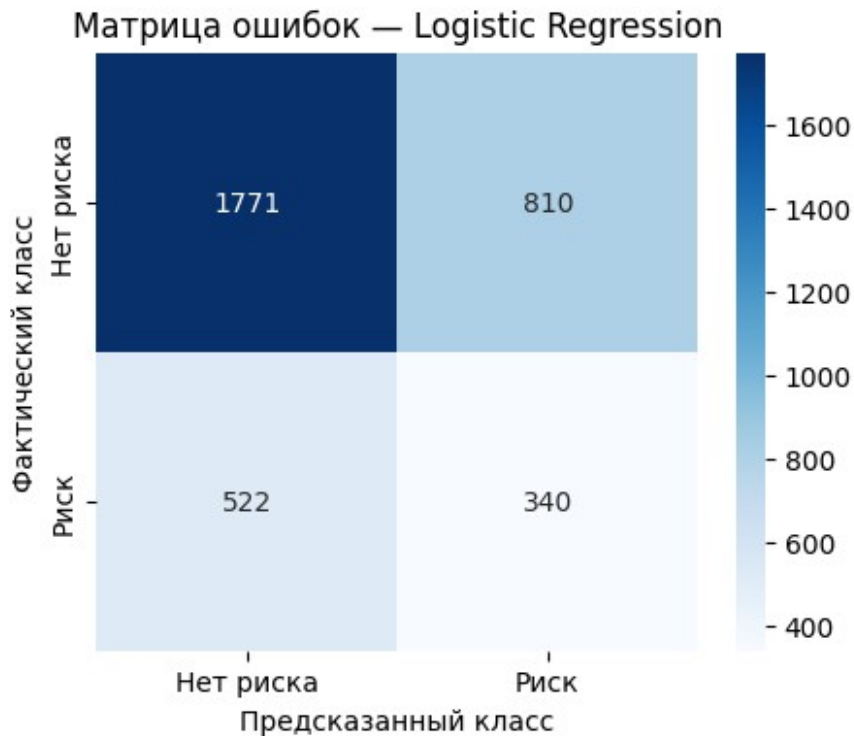
Для класса риска значение Recall составило 0,39, что означает обнаружение около 39% рискованных наблюдений.

Полученные результаты свидетельствуют о наличии более сложных зависимостей между признаками и целевой переменной, которые могут быть недостаточно хорошо описаны линейной моделью.

```
cm_lr = confusion_matrix(y_test, y_pred_lr)

plt.figure(figsize=(5,4))
sns.heatmap(
    cm_lr,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Нет риска', 'Риск'],
    yticklabels=['Нет риска', 'Риск']
)

plt.xlabel('Предсказанный класс')
plt.ylabel('Фактический класс')
plt.title('Матрица ошибок – Logistic Regression')
plt.show()
```



Матрица ошибок показывает, что модель значительно лучше распознаёт наблюдения без риска, чем наблюдения с риском.

Из 862 фактических случаев риска модель корректно определила только 340 случаев, а 522 случая были отнесены к классу без риска.

Это подтверждает ограниченную эффективность логистической регрессии для рассматриваемой задачи и необходимость проверки более сложных алгоритмов машинного обучения.

5.2 Random Forest

```
X_rf = X.copy()
le_station = LabelEncoder()
X_rf['Буровой станок'] = le_station.fit_transform(
    X_rf['Буровой станок']
)
```

Для алгоритма Random Forest категориальный признак «Буровой станок» был преобразован в числовой формат с помощью LabelEncoder. В отличие от логистической регрессии, алгоритмы на основе деревьев решений не требуют стандартизации признаков.

```
X_train_rf, X_test_rf, y_train_rf, y_test_rf = train_test_split(
    X_rf,
    y,
```

```

    test_size=0.2,
    random_state=42,
    stratify=y
)

rf_model = RandomForestClassifier(
    class_weight='balanced',
    random_state=42
)

rf_model.fit(X_train_rf, y_train_rf)

RandomForestClassifier(class_weight='balanced', random_state=42)

y_pred_rf = rf_model.predict(X_test_rf)

y_proba_rf = rf_model.predict_proba(X_test_rf)[:, 1]

rf_metrics = {
    'Модель': 'Random Forest',
    'Accuracy': round(accuracy_score(y_test_rf, y_pred_rf), 3),
    'Precision': round(precision_score(y_test_rf, y_pred_rf), 3),
    'Recall': round(recall_score(y_test_rf, y_pred_rf), 3),
    'F1-score': round(f1_score(y_test_rf, y_pred_rf), 3),
    'ROC-AUC': round(roc_auc_score(y_test_rf, y_proba_rf), 3)
}

print(classification_report(y_test_rf, y_pred_rf))

```

	precision	recall	f1-score	support
0	0.75	0.76	0.76	2581
1	0.26	0.25	0.26	862
accuracy			0.64	3443
macro avg	0.51	0.51	0.51	3443
weighted avg	0.63	0.64	0.63	3443

```

cm_rf = confusion_matrix(y_test_rf, y_pred_rf)

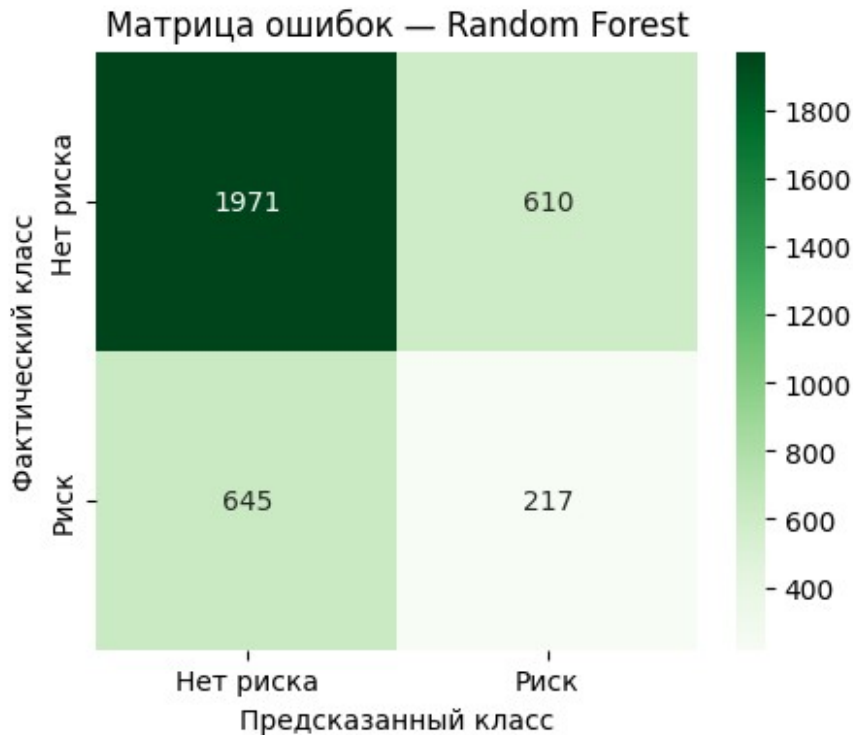
plt.figure(figsize=(5,4))

sns.heatmap(
    cm_rf,
    annot=True,
    fmt='d',
    cmap='Greens',
    xticklabels=['Нет риска', 'Риск'],
    yticklabels=['Нет риска', 'Риск']
)

```

```
plt.xlabel('Предсказанный класс')
plt.ylabel('Фактический класс')
plt.title('Матрица ошибок — Random Forest')

plt.show()
```



Модель Random Forest достигла точности 63,5% при ROC-AUC 0,522. Матрица ошибок показывает, что модель лучше распознаёт наблюдения без риска, чем случаи снижения производительности. Для повышения качества прогнозирования целесообразно рассмотреть более сложные алгоритмы машинного обучения.

5.3 CatBoost

CatBoost представляет собой алгоритм градиентного бустинга над деревьями решений, разработанный специально для работы с табличными данными. Одним из преимуществ алгоритма является возможность непосредственной обработки категориальных признаков без предварительного кодирования.

```
X_cb = X.copy()

X_train_cb, X_test_cb, y_train_cb, y_test_cb = train_test_split(
    X_cb,
    y,
    test_size=0.2,
    random_state=42,
```

```

    stratify=y
)
cat_features = [
    'Буровой станок',
    'Номер смены'
]
cb_model = CatBoostClassifier(
    iterations=300,
    depth=6,
    learning_rate=0.05,
    auto_class_weights='Balanced',
    random_seed=42,
    verbose=False
)
cb_model.fit(
    X_train_cb,
    y_train_cb,
    cat_features=cat_features
)

CatBoostClassifier(auto_class_weights='Balanced', depth=6,
iterations=300, learning_rate=0.05, random_seed=42, verbose=False)

y_pred_cb = cb_model.predict(X_test_cb)
y_proba_cb = cb_model.predict_proba(X_test_cb)[: , 1]
cb_metrics = {
    'Модель': 'CatBoost',
    'Accuracy': round(accuracy_score(y_test_cb, y_pred_cb), 3),
    'Precision': round(precision_score(y_test_cb, y_pred_cb), 3),
    'Recall': round(recall_score(y_test_cb, y_pred_cb), 3),
    'F1-score': round(f1_score(y_test_cb, y_pred_cb), 3),
    'ROC-AUC': round(roc_auc_score(y_test_cb, y_proba_cb), 3)
}
print(classification_report(y_test_cb, y_pred_cb))

```

	precision	recall	f1-score	support
0	0.79	0.55	0.65	2581
1	0.29	0.57	0.39	862
accuracy			0.55	3443
macro avg	0.54	0.56	0.52	3443
weighted avg	0.67	0.55	0.58	3443


```

cm_cb = confusion_matrix(y_test_cb, y_pred_cb)

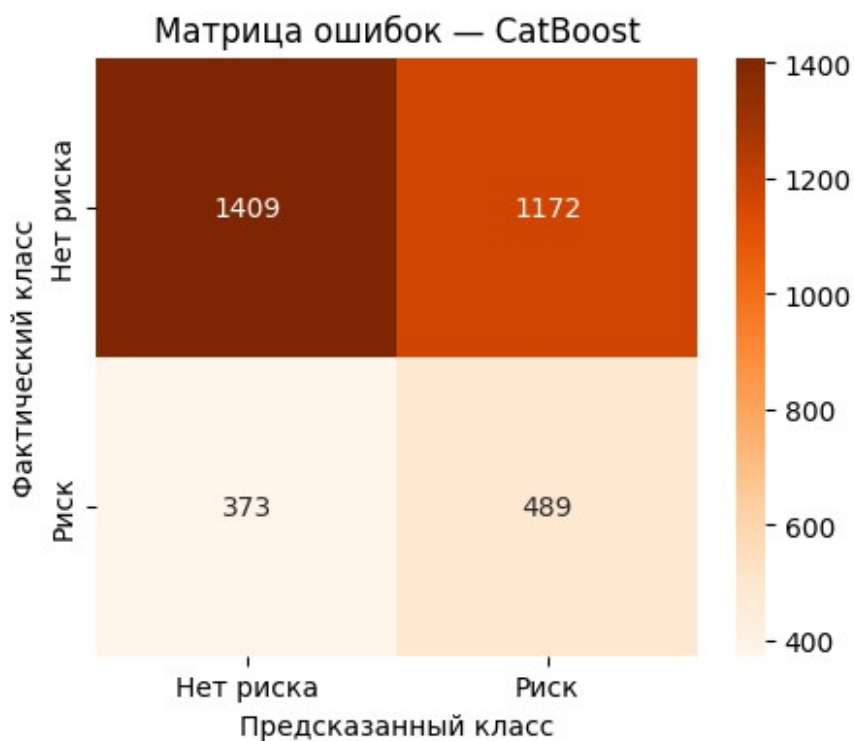
plt.figure(figsize=(5,4))

sns.heatmap(
    cm_cb,
    annot=True,
    fmt='d',
    cmap='Oranges',
    xticklabels=['Нет риска', 'Риск'],
    yticklabels=['Нет риска', 'Риск']
)

plt.xlabel('Предсказанный класс')
plt.ylabel('Фактический класс')
plt.title('Матрица ошибок — CatBoost')

plt.show()

```



Модель CatBoost продемонстрировала умеренное качество классификации (Accuracy = 0.551, ROC-AUC = 0.573). При этом модель выявила 56.7% наблюдений, относящихся к классу риска, что подтверждает наличие связи между используемыми признаками и вероятностью снижения производительности. Полученные результаты показывают возможность применения модели для выявления потенциально рискованных наблюдений, однако для повышения качества прогнозирования требуется дальнейшее совершенствование модели и расширение набора признаков.

5.4 Сравнение моделей

```
results = pd.DataFrame([
    logreg_metrics,
    rf_metrics,
    cb_metrics
])
```

results

	Модель	Accuracy	Precision	Recall	F1-score	ROC-AUC
0	Logistic Regression	0.613	0.296	0.394	0.338	0.544
1	Random Forest	0.635	0.262	0.252	0.257	0.522
2	CatBoost	0.551	0.294	0.567	0.388	0.573

По результатам первичного сравнения моделей наилучшие значения Recall, F1-score и ROC-AUC показала модель CatBoost.

Для данной задачи наиболее значимыми являются метрики Recall и F1-score. Основной целью модели является выявление наблюдений с риском снижения производительности, поэтому особенно важно минимизировать количество пропущенных рискованных случаев. Метрика Recall показывает долю обнаруженных рискованных наблюдений, а F1-score позволяет оценить баланс между полнотой выявления риска и точностью прогнозирования.

Модель CatBoost была выбрана для дальнейшего улучшения, поскольку продемонстрировала наилучшую способность выявлять рискованные наблюдения и показала наибольшее значение ROC-AUC среди рассмотренных алгоритмов.

6 Улучшение качества модели CatBoost

6.1 Генерация дополнительных признаков

Для повышения качества модели была выполнена генерация дополнительных признаков.

Из даты смены были выделены месяц, квартал и день года. Эти признаки позволяют учитывать возможную сезонность и временные особенности работы бурового оборудования.

Также были добавлены признаки взаимодействия простоев с коэффициентами КИО, КНП и КТГ. Такой подход позволяет учитывать не только сам факт простоя, но и его влияние с учётом общей эффективности и технического состояния оборудования.

```
df_fe = df.copy()

df_fe['Месяц'] = df_fe['Дата смены'].dt.month
df_fe['Квартал'] = df_fe['Дата смены'].dt.quarter
df_fe['День_года'] = df_fe['Дата смены'].dt.dayofyear
```

```

df_fe['Простой_КИО'] = df_fe['Простой, ч'] * df_fe['КИО']
df_fe['Простой_КНП'] = df_fe['Простой, ч'] * df_fe['КНП']
df_fe['Простой_КТГ'] = df_fe['Простой, ч'] * df_fe['КТГ']

features_fe = [
    'Буровой станок',
    'Номер смены',
    'КИО',
    'КНП',
    'КТГ',
    'Простой, ч',
    'Месяц',
    'Квартал',
    'День_года',
    'Простой_КИО',
    'Простой_КНП',
    'Простой_КТГ'
]

target = 'risk'

X_fe = df_fe[features_fe]
y_fe = df_fe[target]

X_train_fe, X_test_fe, y_train_fe, y_test_fe = train_test_split(
    X_fe,
    y_fe,
    test_size=0.2,
    random_state=42,
    stratify=y_fe
)

cat_features_fe = [
    'Буровой станок',
    'Номер смены',
    'Месяц',
    'Квартал'
]

cb_fe_model = CatBoostClassifier(
    iterations=300,
    depth=6,
    learning_rate=0.05,
    auto_class_weights='Balanced',
    loss_function='Logloss',
    eval_metric='AUC',
    random_seed=42,
    verbose=False
)

cb_fe_model.fit(

```

```

X_train_fe,
y_train_fe,
cat_features=cat_features_fe
)

CatBoostClassifier(auto_class_weights='Balanced', depth=6,
eval_metric='AUC', iterations=300, learning_rate=0.05,
loss_function='Logloss', random_seed=42, verbose=False)

y_pred_fe = cb_fe_model.predict(X_test_fe)
y_proba_fe = cb_fe_model.predict_proba(X_test_fe)[: , 1]

cb_fe_metrics = {
    'Модель': 'CatBoost + Feature Engineering',
    'Accuracy': round(accuracy_score(y_test_fe, y_pred_fe), 3),
    'Precision': round(precision_score(y_test_fe, y_pred_fe), 3),
    'Recall': round(recall_score(y_test_fe, y_pred_fe), 3),
    'F1-score': round(f1_score(y_test_fe, y_pred_fe), 3),
    'ROC-AUC': round(roc_auc_score(y_test_fe, y_proba_fe), 3)
}

pd.DataFrame([cb_fe_metrics])

```

	Модель	Accuracy	Precision	Recall	F1-score
score \					
0	CatBoost + Feature Engineering	0.569	0.316	0.618	
					0.418
	ROC-AUC				
0		0.618			

Добавление дополнительных признаков позволило повысить качество модели по всем ключевым метрикам. Особенно заметно выросли Recall, F1-score и ROC-AUC, что говорит о более качественном выявлении наблюдений с риском снижения производительности.

6.2 Подбор гиперпараметров

Для дополнительного улучшения модели был выполнен подбор гиперпараметров CatBoost с использованием GridSearchCV.

В рамках подбора рассматривались параметры количества итераций, глубины деревьев и скорости обучения. В качестве основной метрики оптимизации использовался F1-score, так как он учитывает баланс между точностью выявления риска и полнотой обнаружения рискованных наблюдений.

```

cb_grid_model = CatBoostClassifier(
    loss_function='Logloss',
    eval_metric='AUC',
    auto_class_weights='Balanced',
    random_seed=42,

```

```

        verbose=False
    )

    param_grid = {
        'iterations': [300, 500],
        'depth': [4, 6, 8],
        'learning_rate': [0.03, 0.05, 0.1]
    }

    grid_search = GridSearchCV(
        estimator=cb_grid_model,
        param_grid=param_grid,
        scoring='f1',
        cv=3,
        n_jobs=-1
    )

    grid_search.fit(
        X_train_fe,
        y_train_fe,
        cat_features=cat_features_fe
    )

    grid_search.best_params_

{'depth': 4, 'iterations': 300, 'learning_rate': 0.05}

```

По результатам подбора были выбраны параметры: глубина дерева 4, количество итераций 300 и скорость обучения 0.05.

```

best_cb_model = grid_search.best_estimator_

y_pred_best = best_cb_model.predict(X_test_fe)
y_proba_best = best_cb_model.predict_proba(X_test_fe)[:, 1]

best_cb_metrics = {
    'Модель': 'CatBoost + FE + GridSearch',
    'Accuracy': round(accuracy_score(y_test_fe, y_pred_best), 3),
    'Precision': round(precision_score(y_test_fe, y_pred_best), 3),
    'Recall': round(recall_score(y_test_fe, y_pred_best), 3),
    'F1-score': round(f1_score(y_test_fe, y_pred_best), 3),
    'ROC-AUC': round(roc_auc_score(y_test_fe, y_proba_best), 3)
}

pd.DataFrame([best_cb_metrics])

```

	Модель	Accuracy	Precision	Recall	F1-score
ROC-AUC					
0	CatBoost + FE + GridSearch	0.569	0.316	0.621	0.419
0.619					

6.3 Итоговые результаты

```
pd.DataFrame([
    cb_metrics,
    cb_fe_metrics,
    best_cb_metrics
])
```

	Модель	Accuracy	Precision	Recall	F1-
score \					
0	CatBoost	0.551	0.294	0.567	
0.388					
1	CatBoost + Feature Engineering	0.569	0.316	0.618	
0.418					
2	CatBoost + FE + GridSearch	0.569	0.316	0.621	
0.419					

	ROC-AUC
0	0.573
1	0.618
2	0.619

```
cm_fe_best = confusion_matrix(y_test_fe, y_pred_best)
```

```
plt.figure(figsize=(5,4))
```

```
sns.heatmap(
    cm_fe_best,
    annot=True,
    fmt='d',
    cmap='Purples',
    xticklabels=['Нет риска', 'Риск'],
    yticklabels=['Нет риска', 'Риск']
)
```

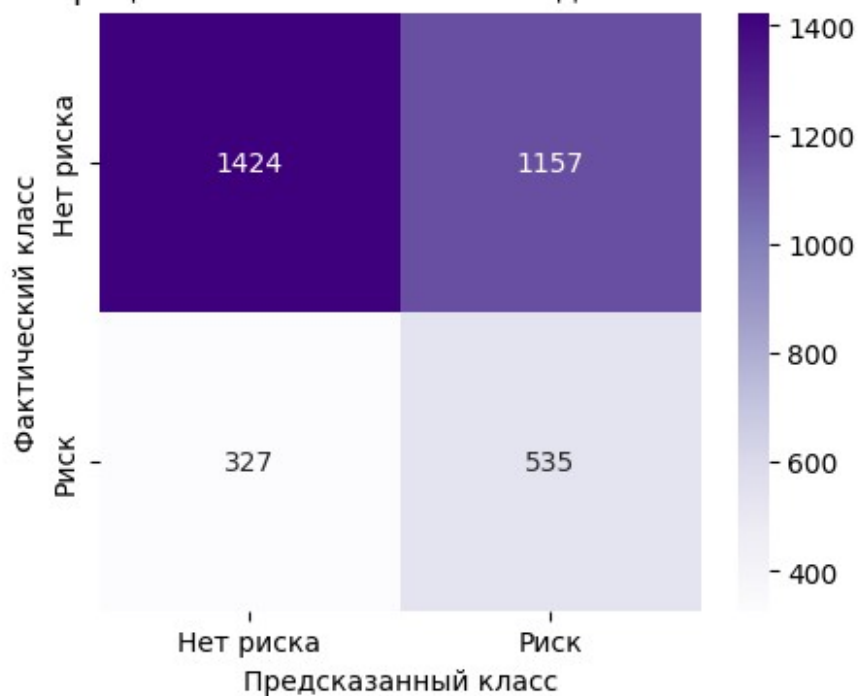
```
plt.xlabel('Предсказанный класс')
```

```
plt.ylabel('Фактический класс')
```

```
plt.title('Матрица ошибок – итоговая модель CatBoost')
```

```
plt.show()
```

Матрица ошибок — итоговая модель CatBoost



Наибольший прирост качества был получен после добавления дополнительных признаков. Подбор гиперпараметров позволил незначительно улучшить результат модели.

В качестве итоговой модели выбрана CatBoost + Feature Engineering + GridSearch, так как она показала лучшие значения Recall, F1-score и ROC-AUC среди рассмотренных вариантов.

Полученная модель может использоваться для формирования рейтинга буровых станков по вероятности риска снижения производительности и дальнейшей визуализации результатов в BI-дашборде.